



Evolutionary Intelligence

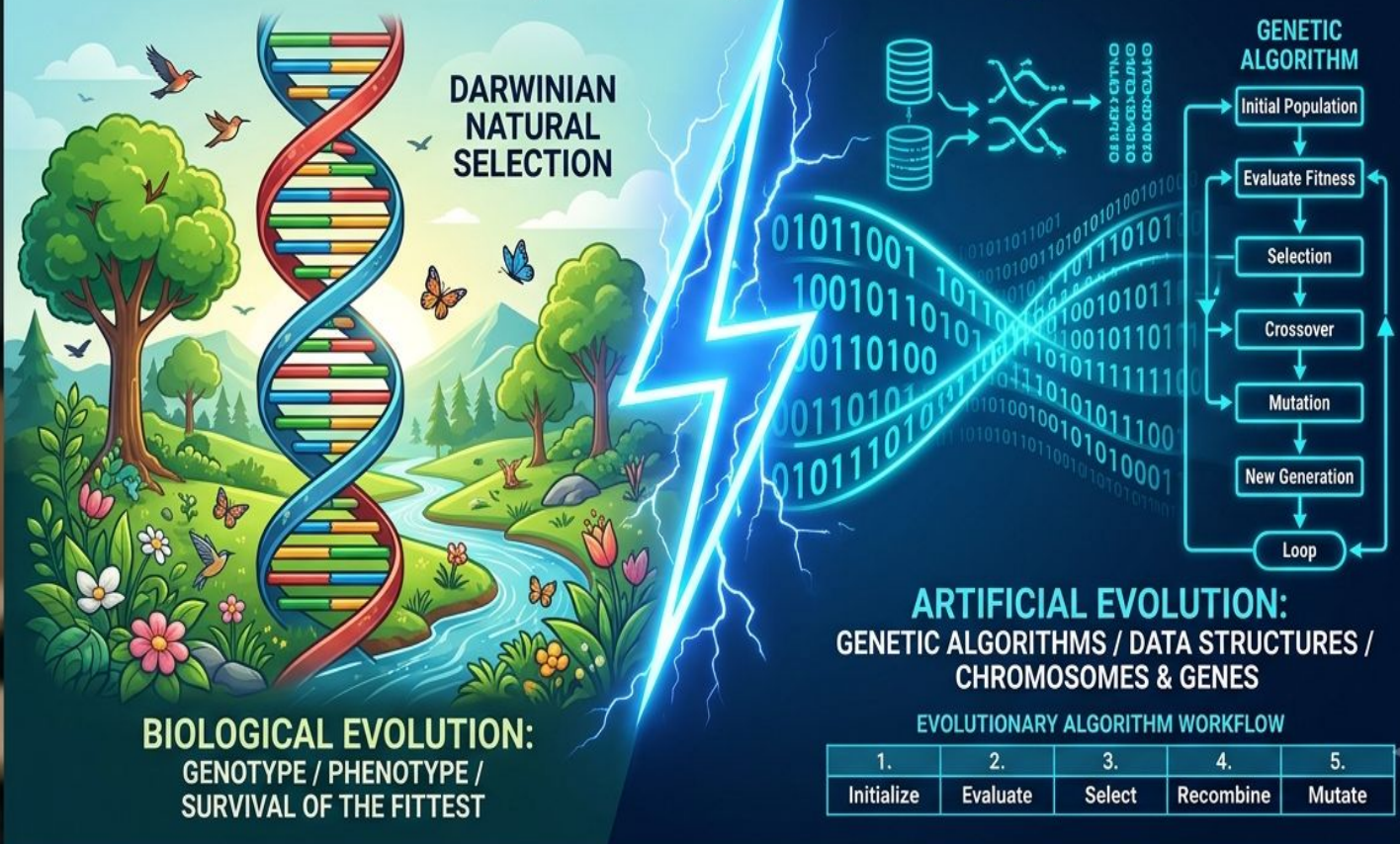
Evolutionary Algorithms: Genetic Algorithms, Selection Operators, Crossover and Mutation, Evolutionary Strategies, Self-Adaptation in Evolutionary Strategies, Types of Evolutionary Strategies, Applications of Evolutionary Intelligence, Optimization Problems, Genetic Programming (GP), Tree-Based Representation in Genetic Programming

Evolutionary intelligence

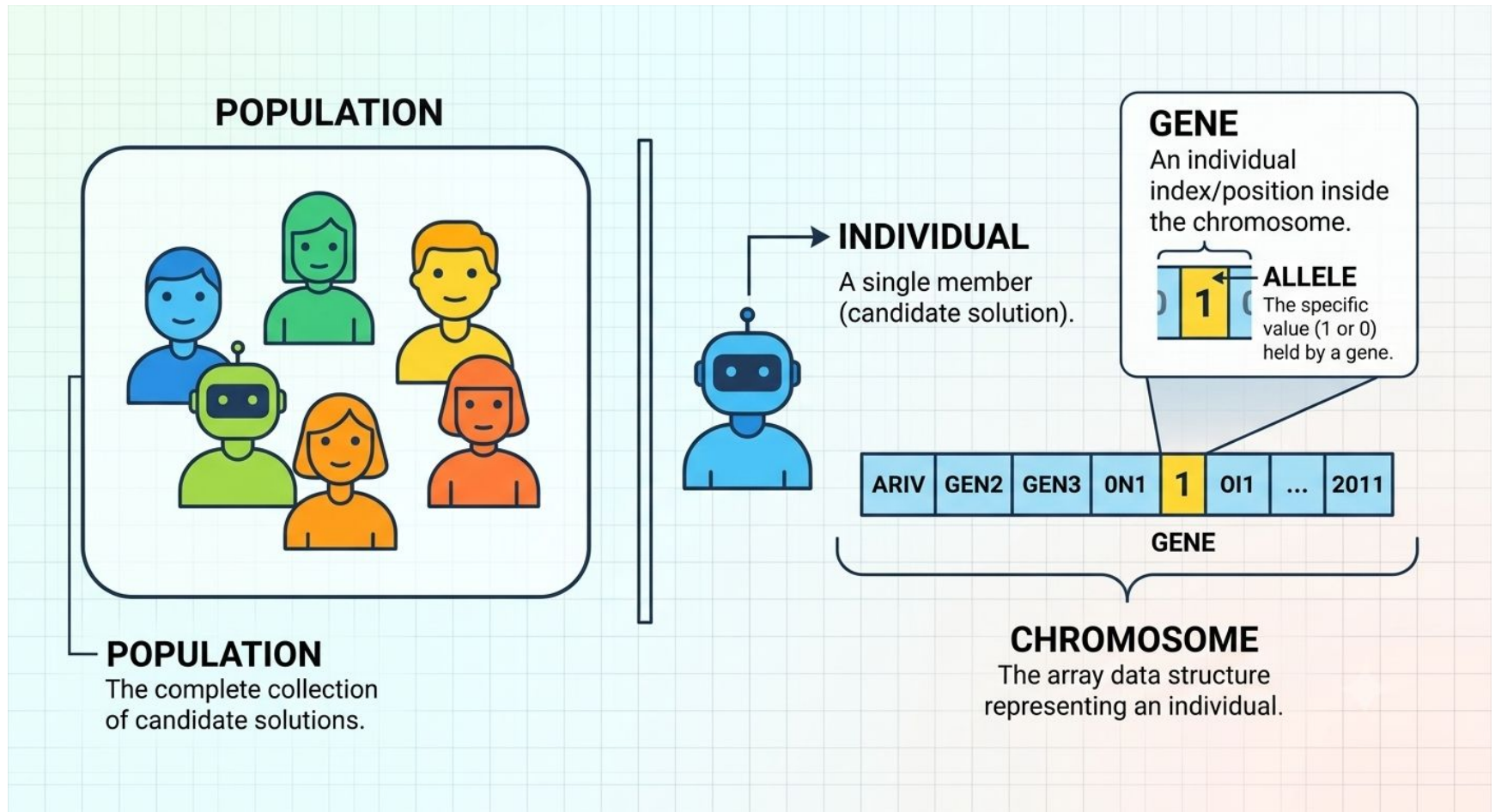
- **Evolutionary intelligence in AI uses nature's rules like survival of the fittest, mutation, and selection to build self-improving computer programs.** It relies on a pool of potential solutions, testing them, and keeping only the best ones to grow smarter over time without human help.



Evolutionary Intelligence & Algorithms



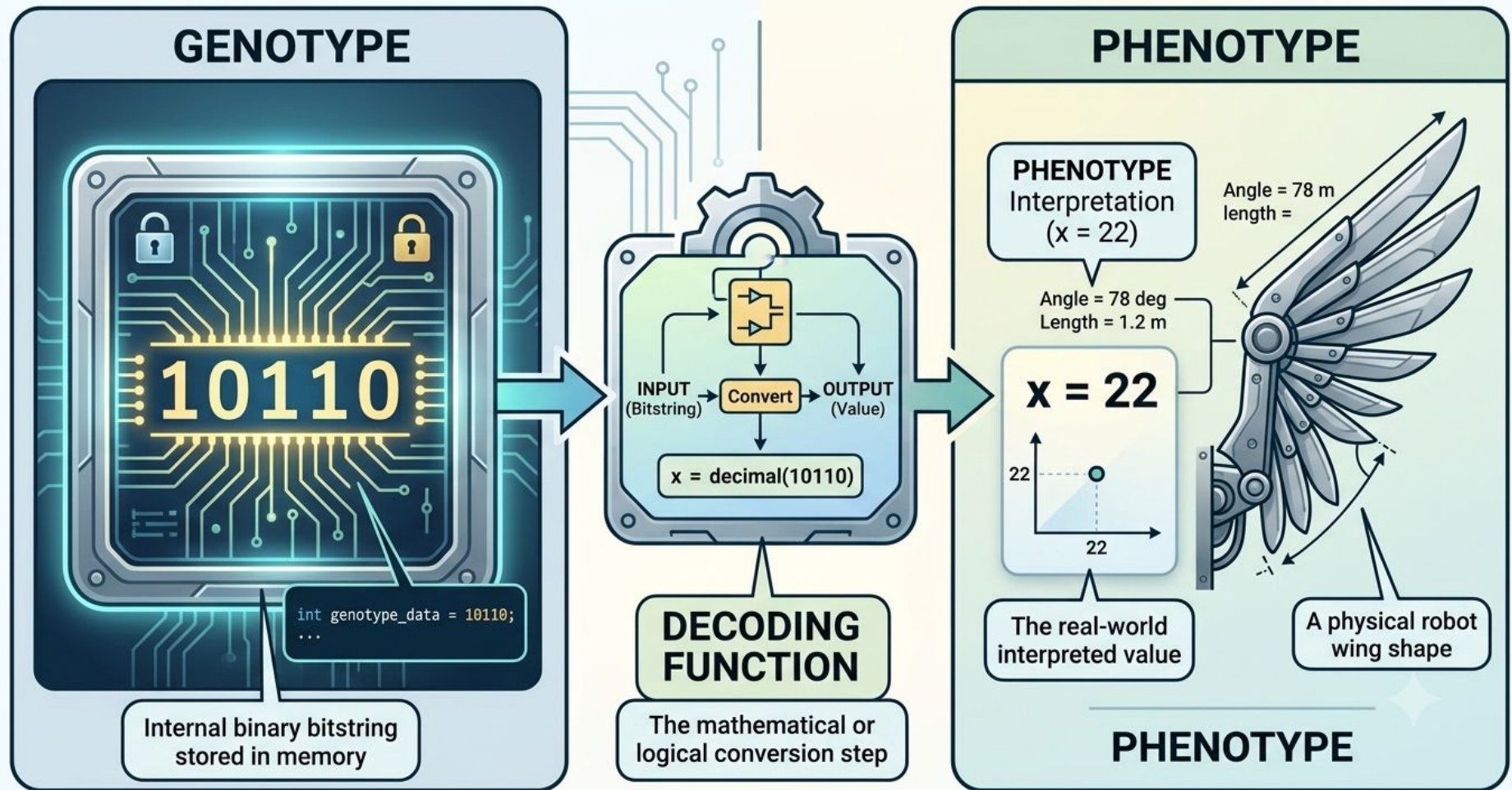
Core Biological Analogy Terms



Core Biological Analogy Terms

- **Population:** A set or collection of candidate solutions (individuals) evaluated at a single time step or generation.
- **Individual (Genome / Candidate Solution):** A single entity within the population representing one complete potential answer to the optimization problem.
- **Chromosome:** The core data structure holding the genetic material of an individual (e.g., a bitstring 10101, an array of integers, or a vector of floating-point numbers).
- **Gene:** The fundamental atomic unit or variable within a chromosome. For example, in the chromosome [1, 0, 1], each bit (1 or 0) is a gene.
- **Allele:** The specific value or state assigned to a gene (e.g., bit value 1 or integer 4).
- **Locus (plural: Loci):** The fixed physical position or index of a gene within a chromosome array.

Genotype vs. Phenotype (Decoding)

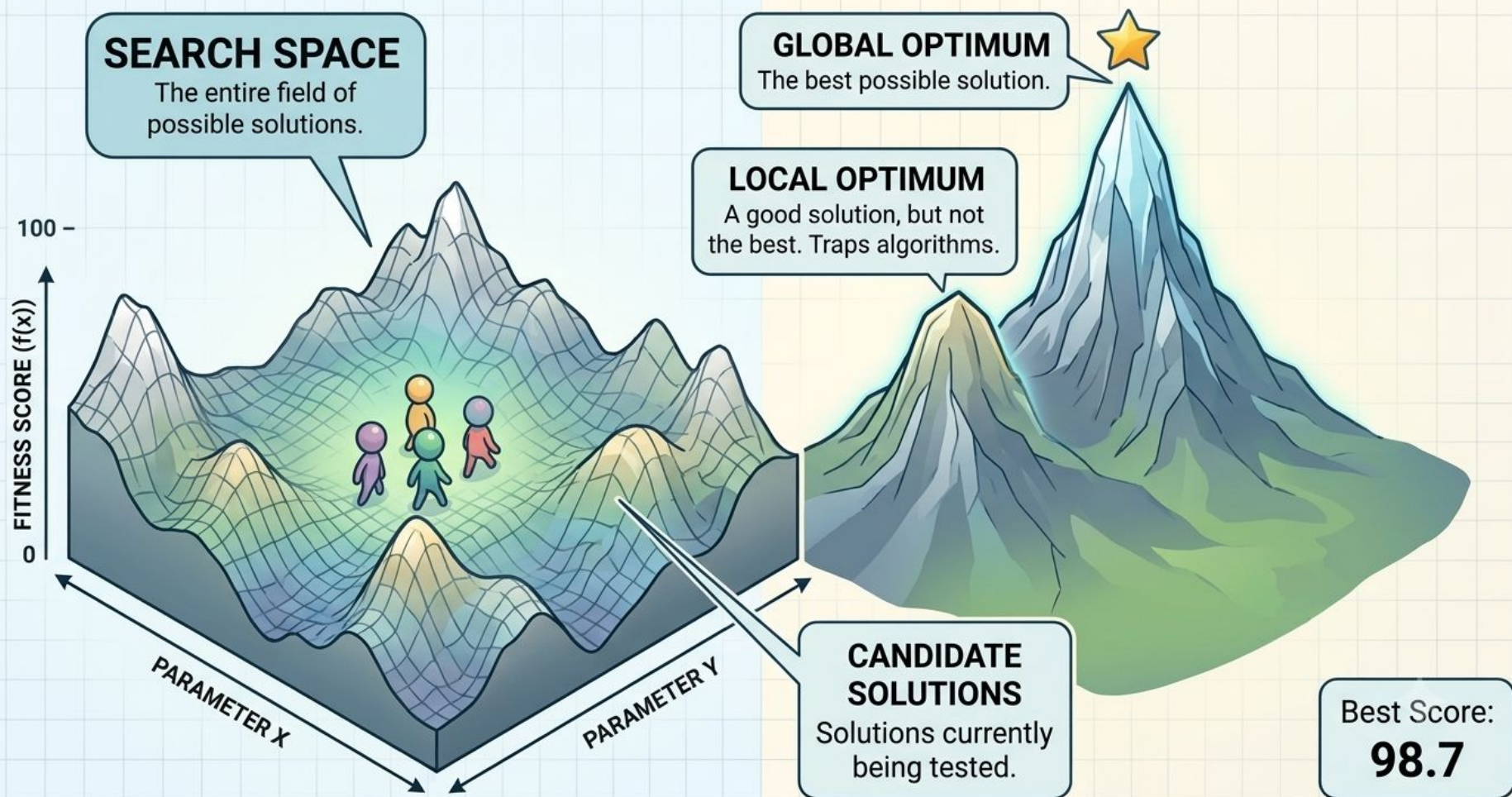


Representation & Decoding Concepts

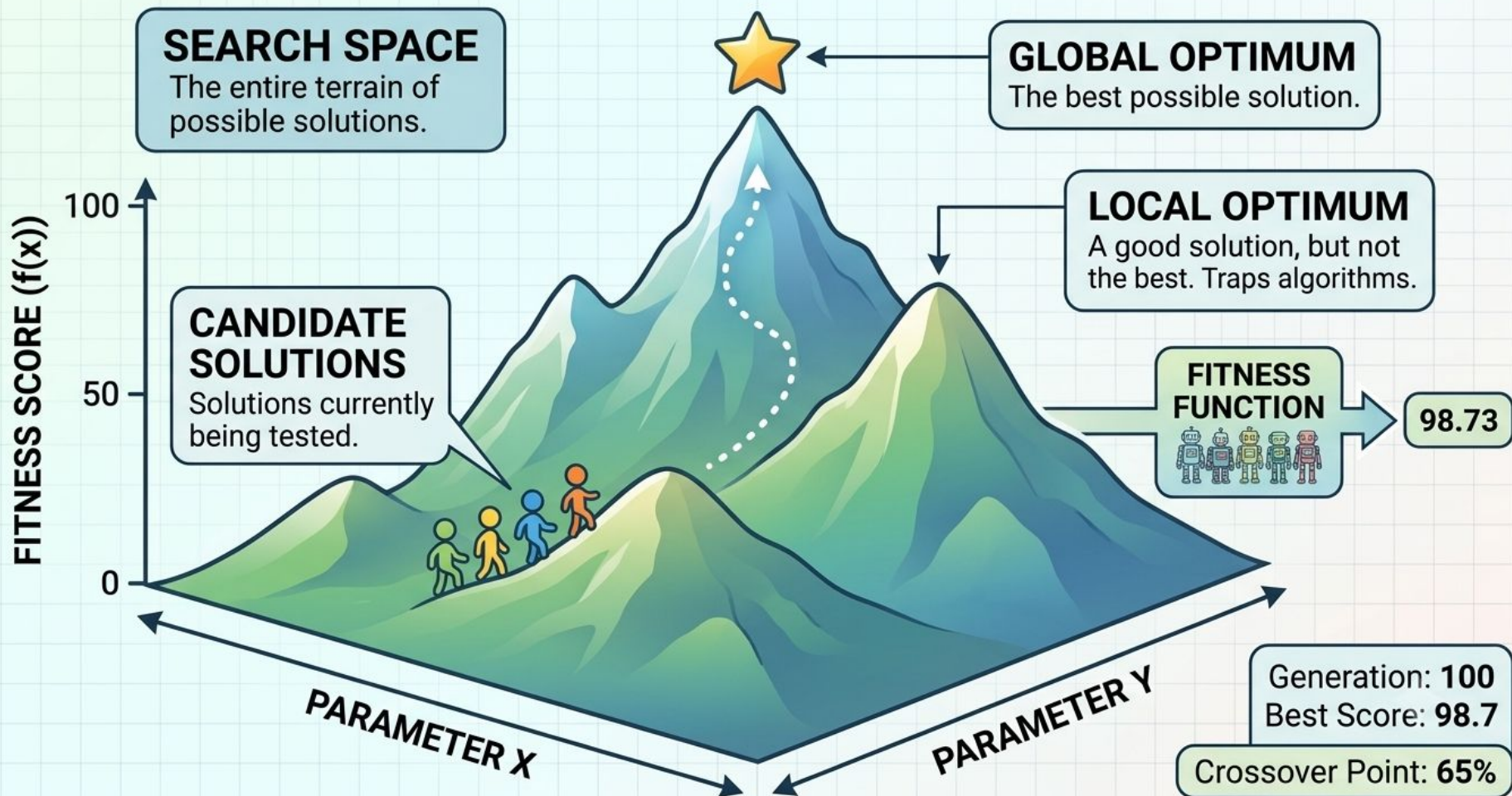
- **Genotype:** The internal, encoded genetic representation of a solution stored in data structures (e.g., binary bitstring 101101).
- **Phenotype:** The real-world decoded meaning or physical trait of the genotype (e.g., decoding binary 101101 to decimal $x = 45$, or converting a route array into an actual delivery path).
- **Encoding / Decoding:**
 - **Encoding:** Transforming a real-world problem state into a genotype data structure.
 - **Decoding:** Translating a genotype back into its phenotype form to evaluate its fitness.



Fitness Function & Search Space



Global and Local Optimum



Evaluation & Optimization

Terminology

- **Fitness Function ($f(x)$):** An objective scoring function that evaluates how 'fit' or optimal a candidate solution (phenotype) is relative to the problem goal.
- **Fitness Score:** The numerical value assigned by the fitness function to rate an individual's performance (higher is typically better).
- **Search Space (Landscape):** The set of all possible valid and invalid candidate solutions for a given problem.
- **Global Optimum:** The best overall solution across the entire search space.
- **Local Optimum:** A sub-optimal peak or valley in the search space that appears best relative to its immediate neighbourhood, potentially trapping optimization algorithms.

Applications of Evolutionary Intelligence

DOMAIN	APPLICATION
Artificial Intelligence & Operations Research	Dynamic Traffic and Route Guidance Systems
Aerospace Engineering & Aerodynamics	AIRCRAFT DESIGN OPTIMIZATION
Healthcare & Biomedical Engineering	PROTEIN STRUCTURE PREDICTION & DRUG DISCOVERY
Cybersecurity & Cryptography	EVOLUTIONARY MALWARE DETECTION

Core operators in an Evolutionary Algorithm

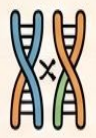
Evaluation

Selection

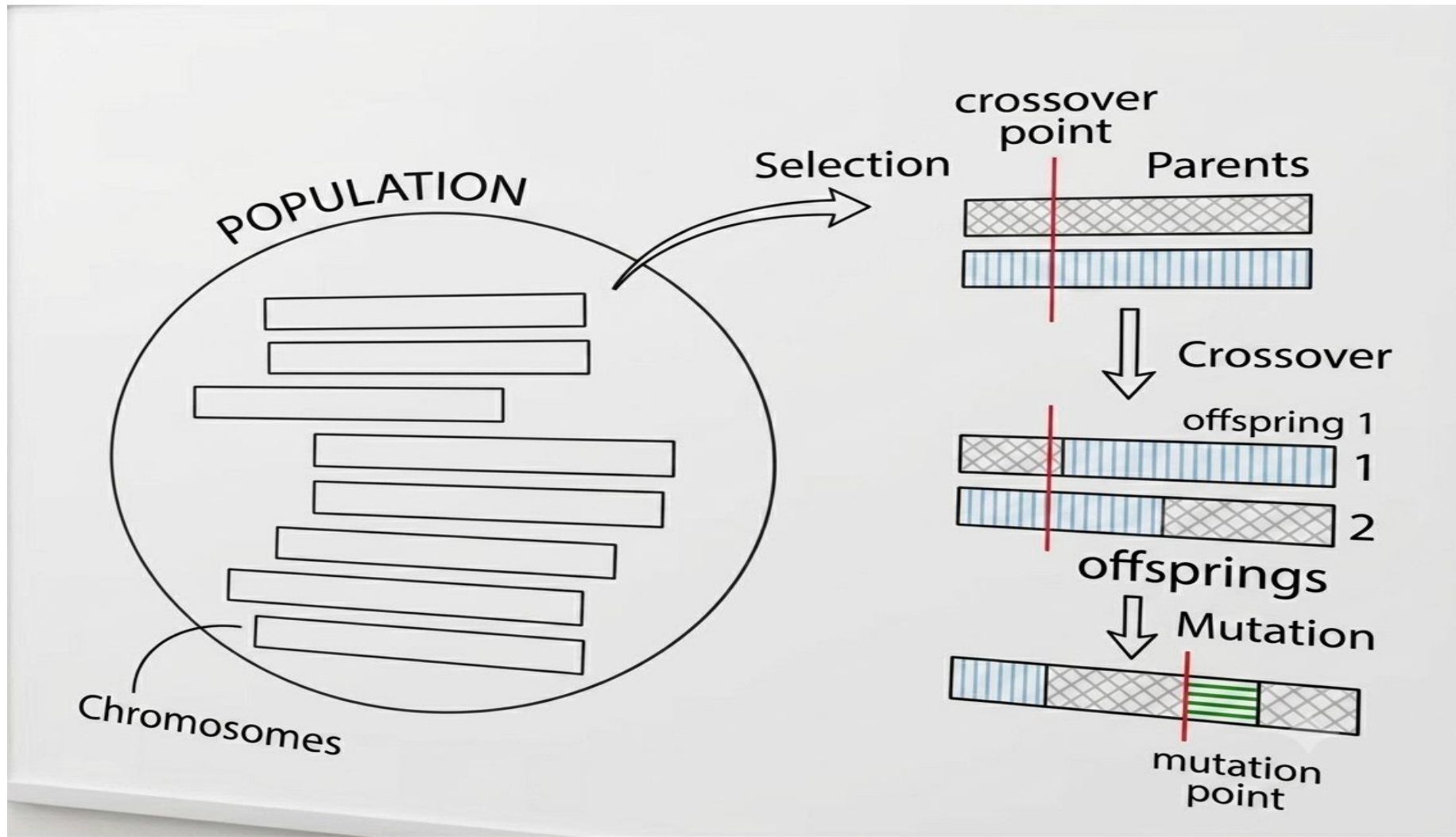
Crossover

Mutation

Operators

Operator (Concept)	What It Is (Function)	How It Works (Mechanism & Effect)
 Evaluation (Fitness Calculation) 	The measuring tape. It evaluates how good or bad a candidate solution is relative to the goal.	You pass a candidate solution into a function, and it outputs a numerical score (a “fitness value”). Key Concept: High fitness = “good solution”. Low fitness = “poor solution”.
 Selection (Survival of the Fittest) 	The filter. It chooses which solutions survive to become parents for the next generation.	Solutions with higher fitness scores are given a higher probability of being picked. Weak solutions are often eliminated. Key Concept: Directs toward better areas of search space by favoring quality.
 Crossover / Recombination (Combining Features) 	The mixer. It takes traits from two parent solutions and combines them to create a new offspring solution.	It chops parts of Parent A and parts of Parent B, then stitches them together. Key Concept: Assumes combining two good solutions might produce an even BETTER solution.
 Mutation (Random Innovation) 	The wild card. It makes small, random changes to a newly created offspring.	With a small probability, it alters one tiny detail of the solution. Key Concept: Maintains diversity and prevents local optima traps.

HOW IT WORKS



Building the Ultimate Sandwich

Imagine an algorithm trying to invent the ultimate gourmet sandwich menu item.

Each **sandwich recipe** is an individual solution.

1. Evaluation (Fitness Calculation)

You make each sandwich and have a panel of judges taste-test them.

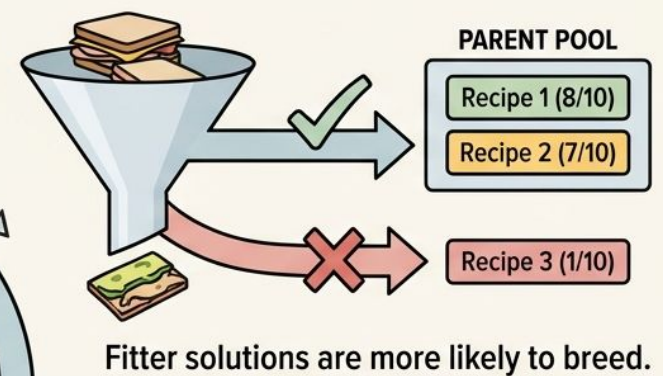
- **Recipe 1** (Turkey, Cheddar, Bacon, Mayo) Score: **8/10**
- **Recipe 2** (Ham, Swiss, Pickles, Mustard) Score: **7/10**
- **Recipe 3** (Tuna, Jam, Peanut Butter, Hot Sauce) Score: **1/10**

Building the Ultimate Sandwich

1 EVALUATION (Taste Test)



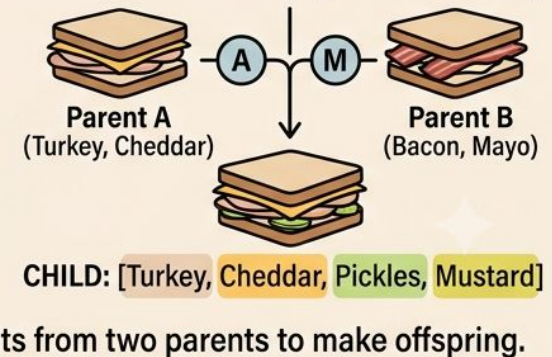
2 SELECTION (Survival of the Fittest)



4 MUTATION (Random Innovation)



CROSSOVER 3 (Combine Traits)



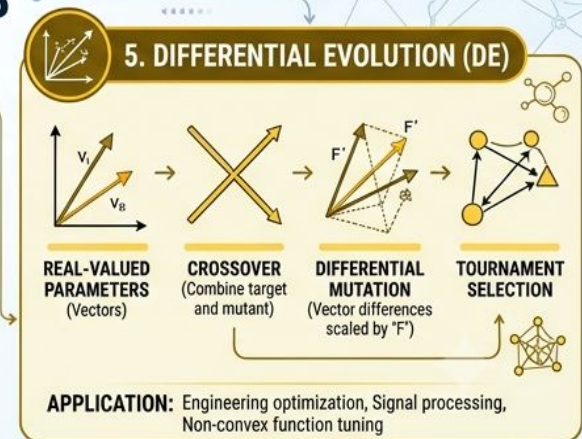
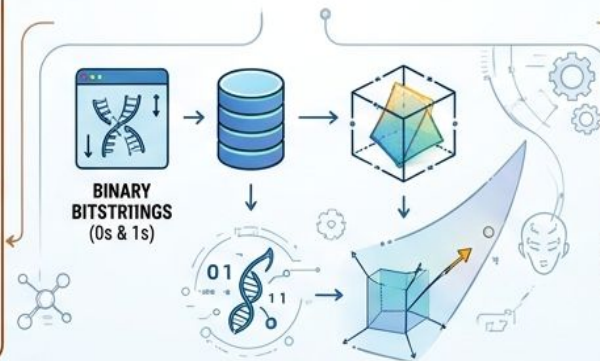
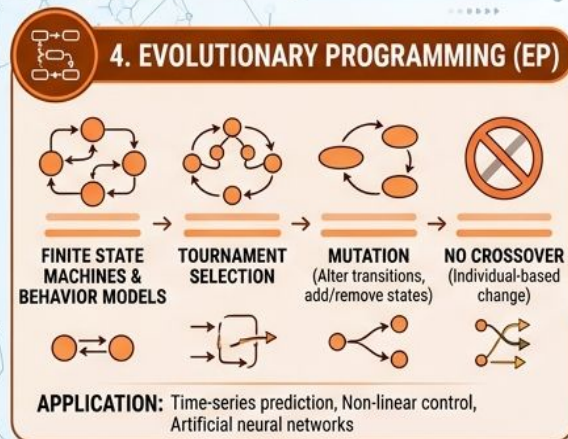
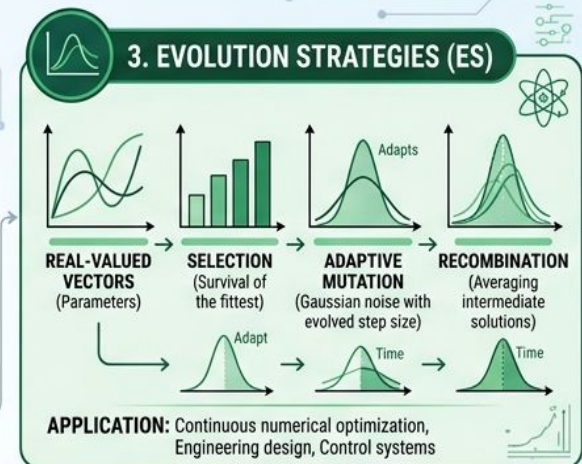
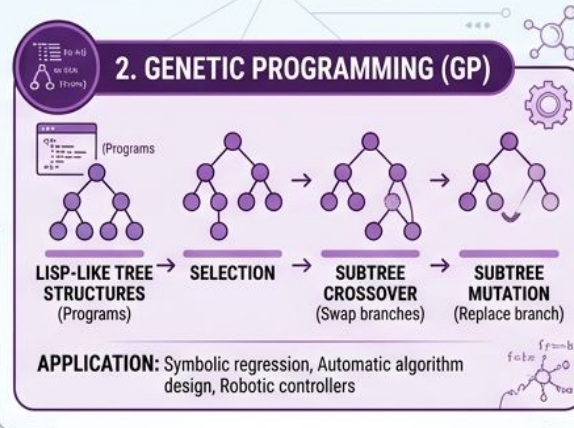
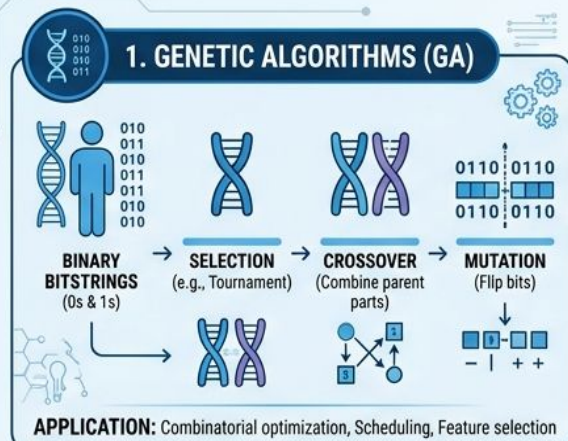
Evolutionary Algorithm

- **Evolutionary algorithms** are a class of algorithms that mimic the genetic improvements of human beings or the natural behaviour of animals. Although there are a variety of evolutionary algorithms, all of them are based on the concept that, when the individuals of a population compete for a limited amount of resources, only the fittest individuals in the population survive.
- **An evolutionary algorithm** is a smart, **nature-inspired technique** used to solve complex optimization problems. It is based on the principles of natural evolution, like survival of the fittest, mutation, and reproduction. In simple terms, when you don't know the best solution to a problem, an evolutionary algorithm helps you find it by testing many possibilities and improving them over time



Types of Evolutionary Algorithms

TYPES OF EVOLUTIONARY ALGORITHMS FROM REPRESENTATION TO SOLUTION



INNOVATION INSPIRED BY BIOLOGICAL EVOLUTION



Genetic Algorithms

- A Genetic Algorithm (GA) is a metaheuristic search and optimization algorithm inspired by Charles Darwin's theory of natural evolution. It belongs to a broader class of algorithms known as Evolutionary Computation.

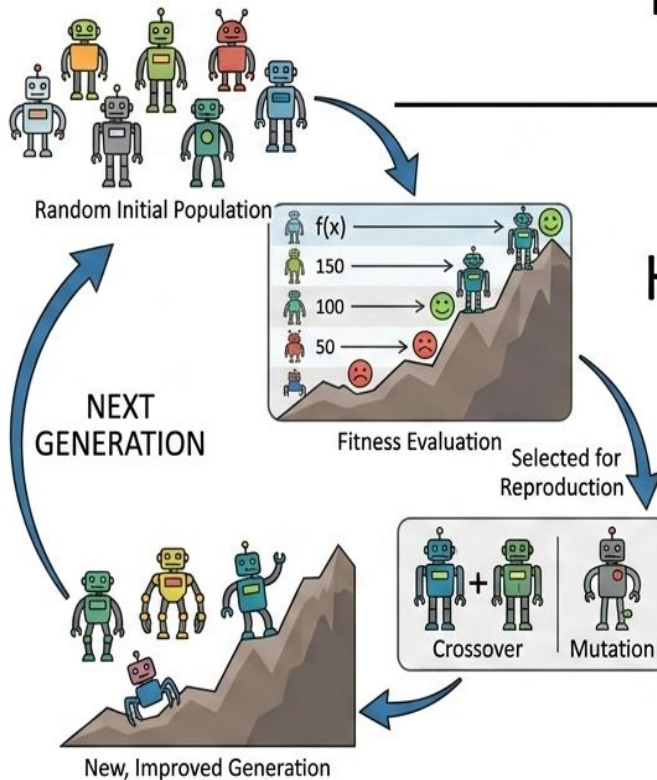
EVOLUTIONARY ALGORITHMS

Genetic Algorithm

Idea: Adaptation is intelligence.

→ Survival of the fittest

How to use this idea for optimization?

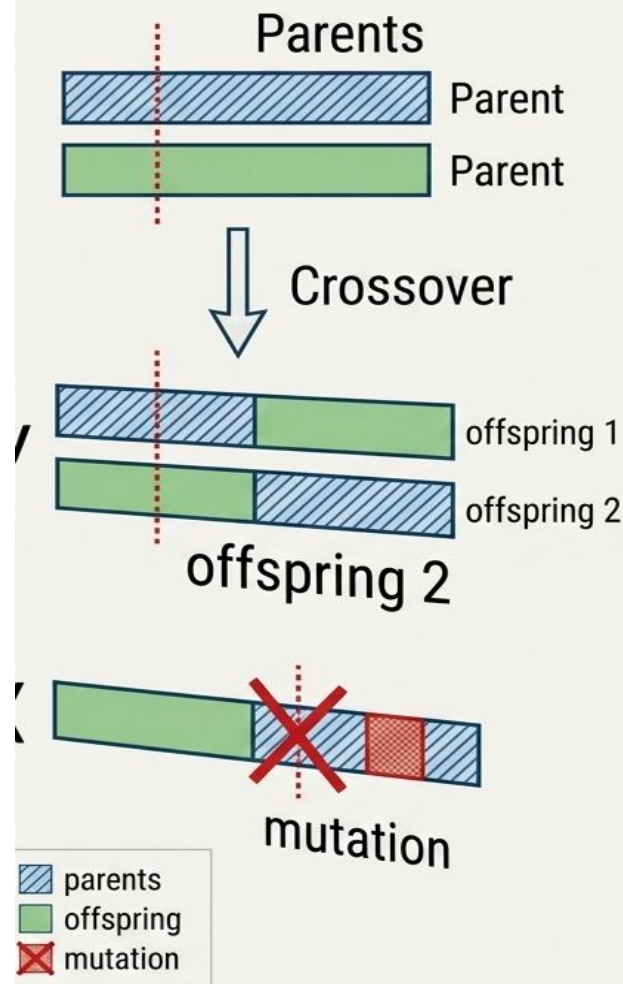


$$\text{Given } f(x) = x_1^2 + \log(x_2) + \frac{\sin(x_3)}{(1-x_4)}$$



GENETIC ALGORITHM FOR OPTIMIZATION: FROM ABSTRACT TO REALITY

ABSTRACT IDEA (Genetic Algorithm Operators)



APPLICATION: Real-Valued Optimization Problem

How to apply this abstract idea to the real-valued optimization problem?

$$f(x) = X_1^2 + \log(X_2) + \frac{\sin(X_3)}{1 - X_4}$$

The diagram shows the mapping of variables X_1 , X_2 , X_3 , and X_4 to binary chromosomes. Arrows point from each variable to a binary sequence:

- X_1 maps to $1\ 1\ 0\ 1\ 1\ 0$
- X_2 maps to $1\ 0\ 0\ 1\ 1\ 1$
- X_3 maps to $1\ 0\ 1\ 1\ 0$
- X_4 maps to $1\ 1\ 1\ 1\ 0$

The final chromosome is a long binary sequence: $1\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1\ 0\ 1\ 0\ 6\ 6\ 1\ 1\ 0$

chromosome

Real-World Applications of Genetic Algorithms



1. Aerospace Engineering: Antenna & Component Design

NASA famously used a Genetic Algorithm to design the **ST5 Spacecraft Antenna**. The evolved, non-intuitive, organic shape outperformed all human-engineered designs in radiation pattern uniformity and efficiency while taking a fraction of the time to design.

2. Logistics & Supply Chain: Vehicle Routing & Timetabling

- **Airline Crew & Flight Scheduling:** Airlines use GAs to schedule flight crews, aircraft maintenance, and gate assignments to minimize costs while adhering to strict regulatory rest periods.
- **Traveling Salesperson Problem (TSP):** Evolving optimal delivery routes for logistics companies to minimize fuel usage and distance across thousands of stops.

Real-World Applications of Genetic Algorithms :

3. Finance & Quantitative Trading

- **Portfolio Optimization:** GAs balance risk versus reward by evolving asset allocation percentages across thousands of market instruments.
- **Algorithmic Rule Generation:** Evolving automated buy/sell trading strategies that dynamically adapt to changing financial market conditions.

4. Game Development & Robotics

- **Pathfinding:** Dynamic 3D path planning for mobile robots navigating unknown or changing environments while minimizing energy consumption.
- **Adaptive Game AI:** Non-Player Characters (NPCs) that adapt their strategies and combat behaviors to counter specific player styles.

5. Molecular Biology & Drug Discovery

- **Protein Structure Prediction:** Evolving 3D molecular conformations to find lowest-energy states, accelerating the initial screening of potential drug compounds.

Core Characteristics Breakdown

1. Representation (Genotype)

- **Discrete / Binary Strings (GA):** Encodes choices as bit strings or ordered lists. Best when decisions are categorical or combinatorial.
- **Syntax Trees (GP):** Nodes represent functions/operators and leaves represent variables/constants. Ideal for evolving executable expressions.
- **Continuous Real Vectors (ES & DE):** Encodes candidate solutions directly as R^2 vectors of floating-point numbers. Ideal for continuous engineering parameters.
- **Phenotypes / State Machines (EP):** Focuses on behavioral outcomes or transition matrices rather than direct low-level gene encodings.

Core Characteristics Breakdown

2. Search Mechanics (Crossover vs. Mutation)

- **Crossover-Dominant (GA, GP):** Relies on recombining sub-solutions ("building blocks") from two parent solutions to discover optimal global structures.
- **Differential Mutation (DE):** Uses the vector difference between randomly sampled population members $V = (X_1 + F \cdot (X_2 - X_3))$ to scale and orient search steps dynamically.
- **Self-Adaptive Mutation (ES):** Simultaneously evolves both the solution vector X and the step-size parameters σ , enabling the algorithm to tune its own search granularity.
- **Mutation-Only (EP):** Omits crossover entirely, relying on behavioral mutations to generate offspring from individual parents.

Decision Matrix: Which EA to Use?

If your problem involves...	Recommended EA	Why?
Routing, Scheduling, Knapsack	Genetic Algorithm (GA)	Handles discrete permutation constraints efficiently via crossover.
Symbolic Regression, Formula Discovery	Genetic Programming (GP)	Naturally searches tree-structured variable spaces without fixed formulas.
High-dimensional continuous function optimization	Differential Evolution (DE)	Difference vectors maintain search scale across ill-conditioned surfaces.
Continuous engineering design with dynamic noise	Evolution Strategies (ES)	Self-adaptation tunes step sizes dynamically near optima.
Autonomous behavior modeling or prediction	Evolutionary Programming (EP)	Directly mutates finite state transitions without breaking tree topologies.

Optimization

- Optimization is the process of making something—such as a system, design, decision, or algorithm—as effective, efficient, or functional as possible given a set of conditions.
- The 3 Pillars of Any Optimization Problem :
 1. **Objective (The Goal):** What you are trying to **maximize** (e.g., profit, speed, accuracy, efficiency) or **minimize** (e.g., cost, error, time, energy usage).
 2. **Variables (The Levers):** The factors or parameters you can adjust to change the outcome.
 3. **Constraints (The Boundaries):** The strict limits or rules you must work within (e.g., budget, time limits, physical space, legal regulations).

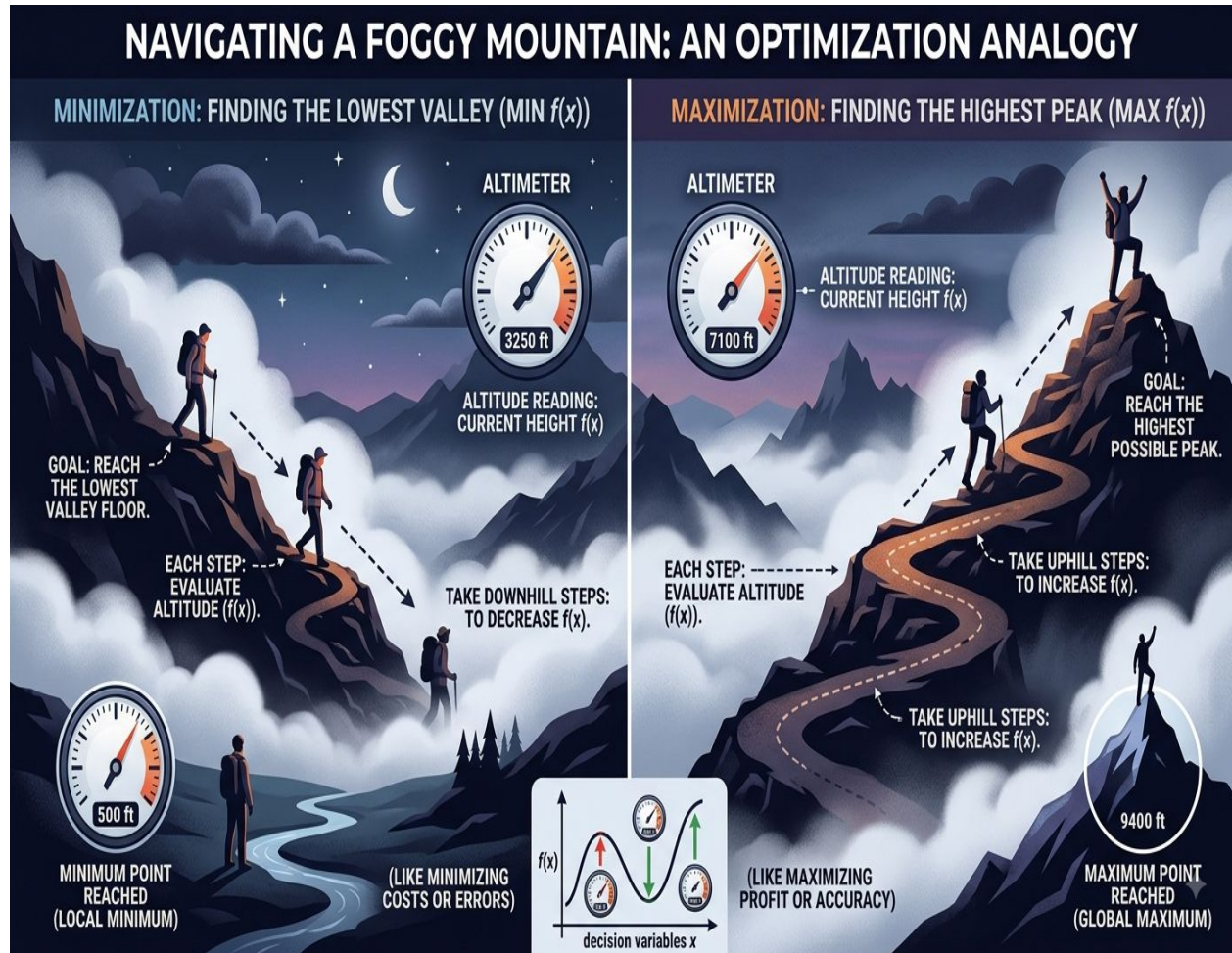
Optimization

Optimization is the mathematical process of **finding the best possible solution to a problem from a set of available alternatives.**

Optimization is simply tweaking your **inputs (x)** to get the best possible result—either making a bad thing as small as possible (minimization) or making a good thing as large as possible (maximization).

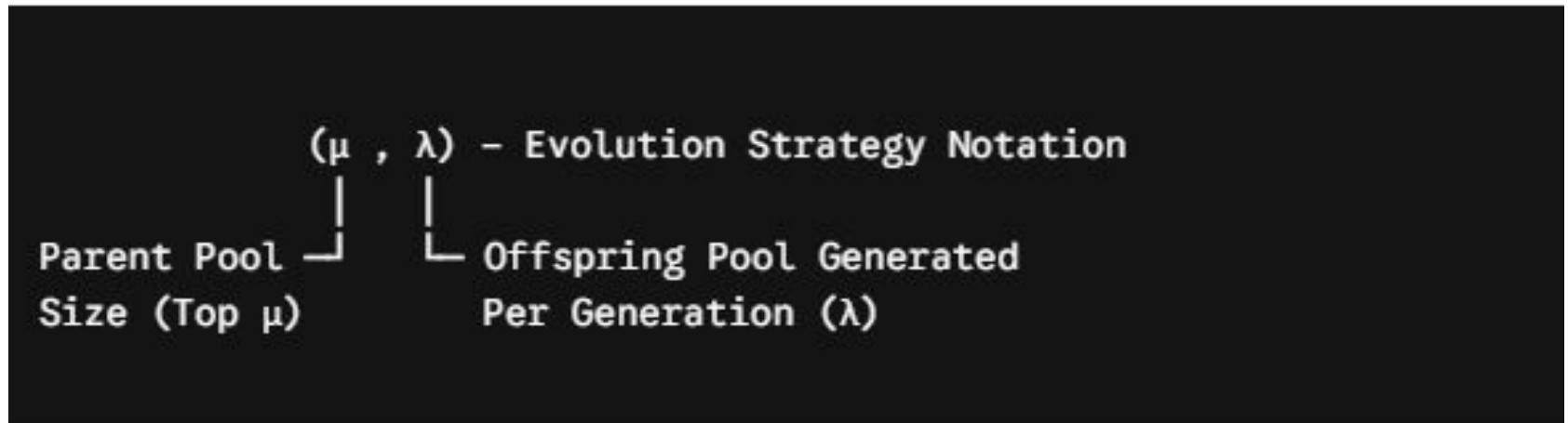
$\min f(x)$ OR $\max f(x)$

Imagine you are stuck on a mountain at night in thick fog. You can't see where you are going; you only have an altimeter that reads your current height $f(x)$.



- **x (Decision Variables):** The controls or knobs you can adjust (e.g., speed, price, temperature).
- **$f(x)$ (Objective Function):** The score or result you care about measuring (e.g., total cost, height, accuracy).
- **x^* :** The absolute best combination of settings you are trying to find.

Evolution Strategies (ES)



μ (Mu): The number of parent solutions selected to breed or survive in each generation.

λ (Lambda): The number of offspring solutions generated from the parents in each generation ($\lambda > \mu$).

Key Terms

Strategy Parameter (σ Sigma): The step size or strength of mutation. It determines how far an individual can "jump" across the solution landscape during mutation.

Self-Adaptation: A unique feature of ES where the mutation step size (σ) is attached directly to an individual's genome and evolves along with the solution parameters.

Exploration vs. Exploitation:

- **Exploration:** Searching broadly across unfamiliar areas of the solution space (high mutation rates).
- **Exploitation:** Fine-tuning and focusing search efforts around a promising known local optimum (small mutation rates).

Can You Answer

If an algorithm generates 100 new child solutions from 20 parent solutions, what are λ & μ ?

$\lambda = 20$ $\mu = 100$

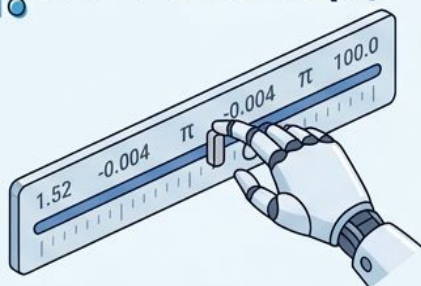
Evolution Strategies

- **Evolution Strategy (ES) is an optimization and machine learning technique** inspired by natural evolution. It belongs to the broader class of Evolutionary Algorithms (EAs).
 - **Evolution Strategies were explicitly engineered for continuous real-valued optimization spaces ($\mathbb{R}^2$),** relying primarily on continuous Gaussian mutation and self-adaptation.
 - **An evolution strategy works with only mutation and no crossover.**
- Traditional evolution strategies use mutation and selection as their main operators. Crossover is optional in these methods.



BREAKING DOWN THE TECHNICAL JARGON: CONTINUOUS OPTIMIZATION IN $\mathbb{R}^n$

1. REAL-VALUED ($\mathbb{R}$)



What it means: Any positive or negative number, including fractions and decimals.

The Opposite (Discrete)



2. CONTINUOUS



What it means: The numbers can change smoothly without gaps. Infinitely many tiny steps between integers.

The Opposite (Discontinuous/Categorical)

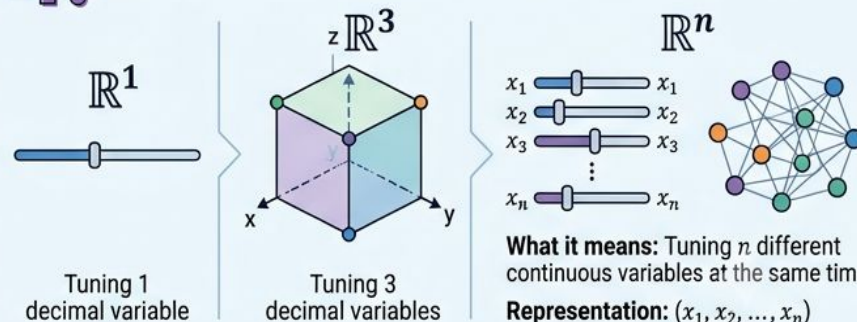


3. OPTIMIZATION SPACE

What it means: The landscape containing every possible combination of settings to test.



4. $\mathbb{R}^n$ (n -DIMENSIONAL REAL SPACE)



SIMPLIFYING HIGH-DIMENSIONAL OPTIMIZATION

The "Dials vs. Switches" Analogy

Light Switches (Discrete Space):

- Each switch can only be **ON (1)** or **OFF (0)**.
- *Genetic Algorithms (GA)* were originally designed for this. They flip bits or swap chunks of sequences.

Volume Dials (Continuous R^n):

- You have 10 smooth knobs, and each knob can be turned to any precise decimal angle (e.g., 42.183 deg).
- **Evolution Strategies (ES) were built for this.** Instead of flipping a switch, ES slightly nudges the dials using continuous Gaussian noise (± 0.05 deg).



The "Dials vs. Switches" Analogy

DISCRETE OPTIMIZATION (GAs)



[1 0 0 1 1]

Flipping Light Switches
(Binary / Combinatorial)

CONTINUOUS OPTIMIZATION (ES)



(4.218 , 12.45° , 0.891)

Tuning Smooth Volume Dials
(Continuous Real Vectors)

Types of ES

Type 1 : (1 + 1)-ES (The Two-Member Strategy) The simplest form. One parent produces one child. If the child is better, it replaces the parent; otherwise, the parent remains.

Type 2 : ($\mu + \lambda$) -ES (Elitist Strategy) μ parents produce λ offspring. The next generation of parents is selected from the combined pool of both parents and offspring.

Type 3: (μ, λ)-ES (Non-Elitist Strategy) μ parents produce λ offspring (where $\lambda > \mu$). The next generation of parents is selected only from the offspring. The old parents are completely discarded, which helps the algorithm escape local optima.

Type 4: CMA-ES (Covariance Matrix Adaptation ES)

Type 5: Natural ES (NES) & Scalable OpenAI ES

(1 + 1)-ES (The Two-Member Strategy)

This is the historical foundation of Evolution Strategies, developed in the 1960s. It is a minimalist, trajectory-based optimization loop.

- **The Core Mechanism:** A single parent solution (1) generates exactly one mutated child solution (1) by adding random Gaussian noise to its parameters.
- **The Filter Rule:** The parent and child go head-to-head. If the child performs better than or equal to the parent, it replaces the parent. Otherwise, the child is discarded, and the parent tries again.

($\mu + \lambda$) -ES (Elitist Strategy)

This strategy scales the search process up by introducing populations of both parents and offspring, applying strict **elitism**.

- **The Core Mechanism:** A pool of μ parent solutions generates a larger pool of λ children via mutation.
- **The Filter Rule:** The **Plus (+)** operator means that both the μ parents and the λ children are thrown into a single competitive pool. The algorithm ranks all of them together and selects the top μ individuals to be the parents for the next generation.
- **Pros & Cons:** It guarantees that the best solution found so far is never lost (perfect elitism). However, if a parent lands on a local peak, it might dominate the pool forever, preventing the algorithm from exploring further.

(μ, λ) -ES (Non Elitist Strategy)

This strategy shifts the survival pressure entirely onto the newer generation to encourage broader exploration.

The Core Mechanism: Just like Type 2, μ parents generate λ children (where λ must be strictly greater than μ)

The Filter Rule: The **Comma** (,) operator means that the original parents are completely discarded at the end of the generation. The next generation of μ parents is chosen **strictly from the λ children pool**, even if the best child is worse than the old parents.

Why it matters: Discarding parents prevents the algorithm from stagnating on local hills, allowing it to cross valleys of poor performance to find global optimal peaks.



(μ, λ) -ES (NON-ELITIST STRATEGY)

EVOLUTIONARY STRATEGIES: SEARCH, ADAPTATION & FORGETTING

WHAT IT IS

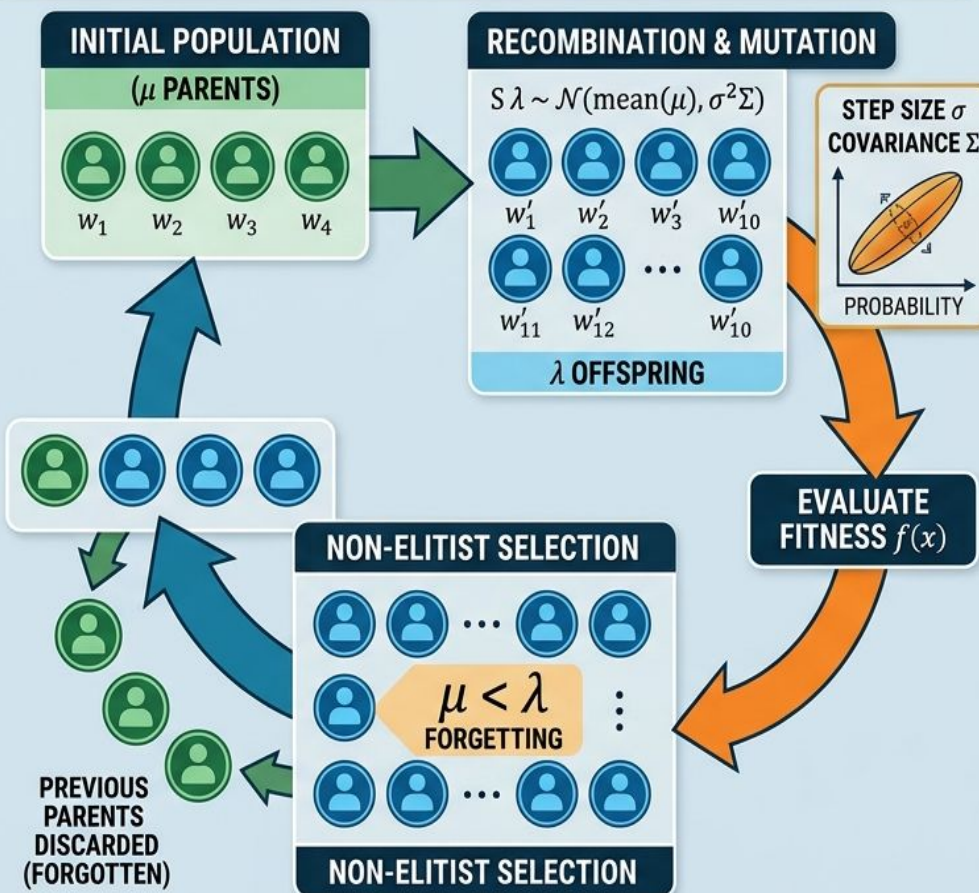


A population-based, stochastic, black-box optimization algorithm where exactly λ offspring are generated from μ parents. It is distinct from Elitist $(\mu+\lambda)$ -ES because previous solutions are forced to be forgotten, and selection only draws from the current λ offspring pool.

MECHANISM



- 1 **START:** Start with μ parents.
- 2 **SAMPLING:** Generate λ offspring.
- 3 **MUTATION:** Shift mutation strength.
- 4 **EVALUATION:** Test λ offspring.
- 5 **SELECTION:** Select the top μ performers strictly from the λ offspring pool to form the next generation. All old parents are discarded.



WHERE IT IS USED



Optimal control, complex engineering design, deep learning hyperparameter tuning, robot locomotion policies, non-convex non-differentiable landscapes.

REALTIME APPLICATION



Reinforcement Learning for Quadruped Locomotion.

On a live robot simulation cluster, the (μ, λ) -ES forgets old outdated joint control policies and adapts to new environment conditions, rapidly selecting the best walking gates from new experimental offspring in a cloud cluster.



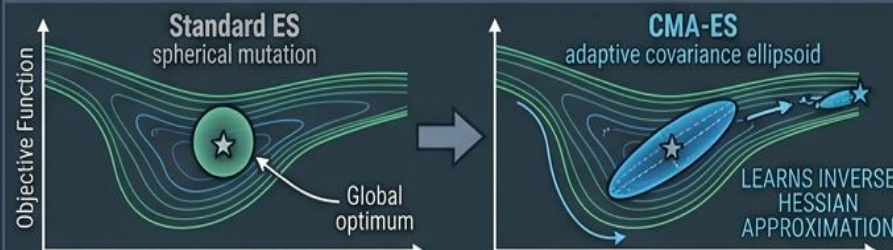
CMA-ES: COVARIANCE MATRIX ADAPTATION EVOLUTION STRATEGY

WHAT IT IS



A population-based, derivative-free, continuous black-box optimization algorithm. CMA-ES dynamically learns and adapts the full covariance matrix C (shape/orientation) and step size σ (speed) of the search distribution.

VISUAL SEARCH MECHANISM



WHERE IT IS USED

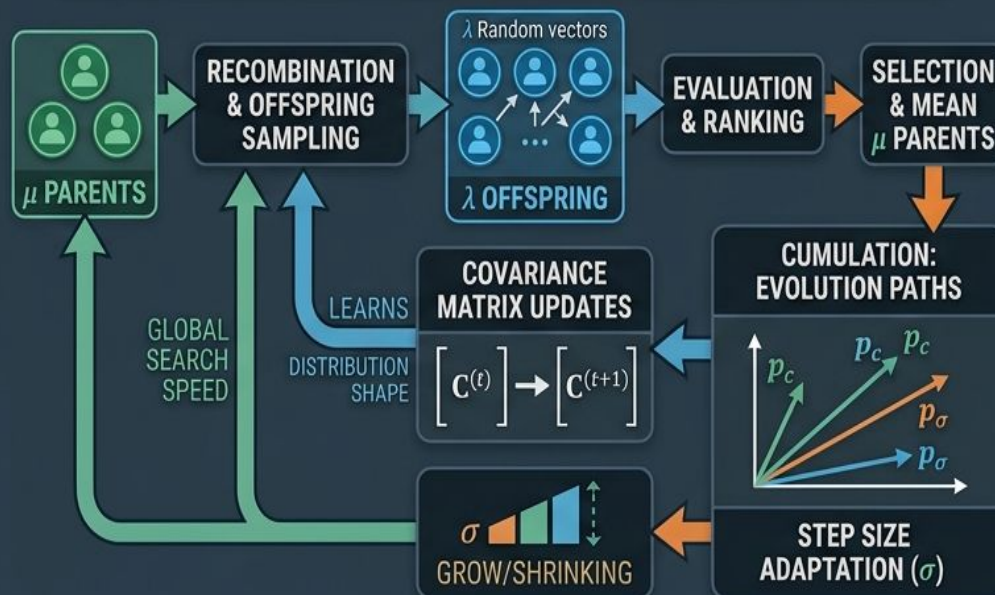


Used in ill-conditioned, non-separable, highly rugged, non-convex objective landscapes. Common in complex engineering design, deep reinforcement learning, automatic hyperparameter search, robotics.

MECHANISM



- 1 **START:** Start with initial parameters $x^{(0)}, \sigma, C$
- 2 **SAMPLING:** Sample λ offspring $y^{(t+1)} \sim \mathcal{N}(x^{(t)}, \sigma^2 C)$
- 3 **EVALUATION:** Rank the λ offspring by fitness.
- 4 **SELECTION:** Select the best μ offspring to form the new mean $x^{(t+1)}$.
- 5 **ADAPTATION:** Update the matrix C and σ using successful evolution paths.



REALTIME APPLICATION



Real-Time Bipedal Locomotion

On a live robot simulation cluster, CMA-ES adapts joint torque control policies ($u_t = \pi(x_t)$) in a dynamic simulation to find stable walking gates. It rapidly selects top policies and covariance shapes by evaluating thousands of simultaneous gait experiments in the cloud.



NATURAL ES (NES) & SCALABLE OPENAI ES: GRADIENT ESTIMATION FOR DEEP RL

WHAT IT IS

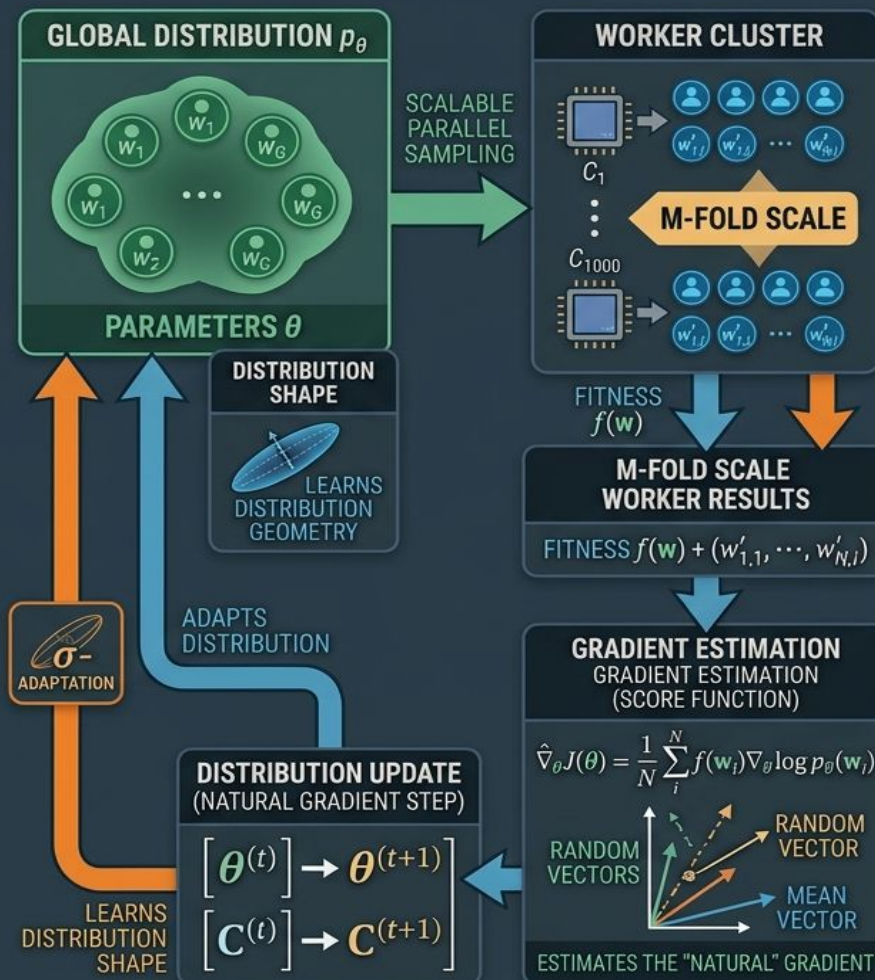


Population-based, derivative-free optimization algorithms for deep neural networks. They treat optimization as maximizing the expected fitness under a probability distribution $p_{\theta}(\mathbf{x})$, learning its parameters θ rather than the raw parameters $\mathbf{x}$. Scalable versions exploit massive parallelization for Deep RL.

MECHANISM



- START:** Initialize distribution parameters θ (e.g., μ, Σ).
- SAMPLING:** Sample thousands of offspring from p_{θ} .
- EVALUATION:** Compute fitness $f(\mathbf{x})$ for all offspring in parallel.
- ESTIMATION:** Use the Score Function to estimate the natural gradient of the expected fitness $J(\theta)$.
- UPDATE:** Update θ in the direction of the estimated natural gradient.



WHERE IT IS USED



Derivative-free optimization in ill-conditioned and noisy Deep Neural Network parameter spaces. Majorly used in Deep Reinforcement Learning (Deep RL), non-differentiable control policies, massive distributed training clusters.

REALTIME APPLICATION



Real-Time Massive Multi-Agent Swarm Control.

On a live simulation cluster with thousands of agents (drones/robots), Scalable OpenAI ES adapts joint control policies for the entire swarm. It rapidly optimizes policies by estimating gradient from thousands of parallel environment experiments in the cloud, adapting to new dynamic obstacles in real time.

Youtube Channel

<https://www.youtube.com/@TechwithPriyanka>

Medium Link :

<https://medium.com/@phajare>